

INDIAN INSTITUTE OF TECHNOLOGY (BHU) VARANASI

Hardware Image Scaler using Verilog

I-Chip PS1 Submission

Team Members

Priyam Kishor Arya

Mithil Damania

Introduction

In this project, we designed and implemented a hardware-based image scaler using Verilog. The objective of the system is to resize an input image. The complete system is synthesizable and controlled using a Finite State Machine (FSM). The image is first loaded into memory, processed pixel by pixel, and finally written back as a resized output image.

Overall Architecture

The architecture of the design is sequential and control driven. Instead of processing multiple pixels in parallel, the system computes one output pixel at a time. This allows reuse of the same arithmetic hardware for each computation, significantly reducing hardware resource usage.

The design consists of:

- Input memory to store the original image
- Output memory to store the resized image
- Coordinate scaling logic
- Interpolation datapath
- Finite State Machine (FSM) controller

Once the `start` signal is asserted, the FSM begins executing the resizing process in a structured and step by step manner.

Coordinate Scaling Method

To resize the image, each output pixel must be mapped to a corresponding location in the input image. Horizontal and vertical scaling factors are calculated as:

$$\text{scale_x} = \frac{W_{IN} \times 256}{W_{OUT}}$$

$$\text{scale_y} = \frac{H_{IN} \times 256}{H_{OUT}}$$

The multiplication by 256 represents 8 fractional bits. Fixed-point arithmetic is used instead of floating-point arithmetic to simplify hardware implementation and ensure synthesizability.

For each output coordinate (x_{out}, y_{out}) , the corresponding input coordinate is calculated using these scaling factors. The integer part determines the base pixel position, while the fractional part is used internally in the computation process.

Finite State Machine Operation

The entire system operates under the control of a Finite State Machine. The FSM ensures that each stage of computation is executed in an organized sequence. The states and their functions are described below:

IDLE: The system remains in this state until the `start` signal is asserted. All counters and control signals are initialized in this state.

CALC_COORDS: In this state, the scaled input coordinates are calculated using the scaling factors. The fixed-point multiplication is performed to determine the mapped input location for the current output pixel.

FETCH_DATA: The required neighboring pixel values are read from input memory. Boundary conditions are checked to prevent out-of-range memory access.

INTERP_X: Intermediate computations along the horizontal direction are performed. This produces partial results required for final pixel computation.

INTERP_Y: Vertical interpolation is completed using the intermediate values from the previous state, producing the final computed pixel value.

WRITE_PIXEL: The final result is clamped between 0 and 255 and written into the output memory at the correct location. Channel counters and pixel counters are updated in this state.

DUMP_FILE: After all pixels are processed, the output memory contents are written to a hex file. The done signal is asserted, and the system returns to the IDLE state.

This state-based approach ensures clean control flow and simplifies debugging during simulation.

Memory Organization

Two large memory arrays are declared in the design. The input memory stores the original image, and the output memory stores the resized image.

Memory indexing is calculated using:

$$\text{index} = (y \times \text{width} + x) \times \text{channels} + \text{channel}$$

The design supports three color channels (RGB). Each channel is processed sequentially to avoid duplication of arithmetic hardware.

Optimization Techniques

Several optimization strategies were used in this implementation:

- Fixed-point arithmetic to reduce hardware complexity.
- Sequential pixel processing to reuse arithmetic resources.
- Channel-wise computation to minimize hardware duplication.
- Extended intermediate bit widths to prevent overflow.
- Boundary checking to avoid invalid memory access.

These techniques ensure that the design remains efficient and synthesizable while handling high-resolution images.

Conclusion

The hardware image scaler was successfully implemented using a structured FSM based architecture. The system is fully synthesizable and resource efficient. Although the sequential processing approach increases overall computation time, it significantly reduces hardware area and complexity.

The final implementation demonstrates a clean, organized, and practical approach to hardware-based image scaling.